

Local Function Stomping Injection

dimanche 8 mars 2026 23:11

Ici le but est de d'écraser une fonction et de la remplacer par notre payload. D'où le terme stomping qui nous rappelle l'action écraser.

Pour la partie local on va devoir créer un processus, créer une fonction qui va se sacrifier, prendre son adresse et injecter notre fonction malveillante.

Les fonctions de la plupart des processus sont invoqué dans les dll. La partie compliqué de cette attaque est de savoir quelle fonction on a récupérer l'adresse. Ce qui veut dire qu'on va devoir écraser une fonction+ venant d'une DLL. Il vaut mieux ne pas choisir des dll du types : ntdll.dll, kernel32.dll et kernelbase.dll.

De plus, une fois que la fonction est écrasé par notre payload, elle ne peut plus être utilisé. Donc vaut mieux choisir une fonction qui est appelé une fois.. (dur). LA logique de ce code serait de comme pour les dll, récupérer l'adresse de cette fonction avec **LoadLibraryA**, récupérer l'adresse avec **GetProcAddress** puis écraser cette fonction avec notre payload.

Quand on est en local on peut juste indiqué les dll qu'on va utiliser dans le code avec un #define.
CE qui nous donne :

Dans cette exo notre target sera la fonction SetupScanFileQueueA :

```
#define SACRIFICIAL_DLL      "setupapi.dll"
#define SACRIFICIAL_FUNC    "SetupScanFileQueueA"
```

On va changer la protection de la fonction avec VirtualProtect

```
BOOL VirtualProtect(
    [in] LPVOID lpAddress,
    [in] SIZE_T dwSize,
    [in] DWORD flNewProtect,
    [out] PDWORD lpflOldProtect
);
```

car il faut que la zone mémoire ou se situe la fonction soit modifiable (RW). Puis on injectera notre payload et on changera la zone mémoire avec RWX pour qu'on puisse exécuter notre payload.

Ce qui nous donne :

```
If(!VirtualProtect()){
Return -1;
```

